

Computational Physics and Data Science Exercise 3 - Linear Regression by Methods of Varying Efficacy

Christian O’Conor
February 8th 2025
Word count: 3139

This study utilized scikit-learn’s linear regression and loss functions to analyse a dataset of free-falling spheres made from materials like iron, titanium, and polycarbonate. The dataset included density, mass, radius, temperature, pressure, height, and time of fall. After cleaning, analyses examined feature correlations, compared loss functions, and assessed gradient descent efficiency. Model validity was tested against an analytical model, revealing insights into material behaviour and regression performance. The study underscored the importance of data preprocessing and method selection for accurate predictive modelling.

1 Introduction

The dataset includes eight features measured in standard units: density, mass, radius, ambient temperature, pressure, height, and time of fall. Density, mass, and radius describe the spheres’ physical properties, while temperature and pressure act as control variables. Height and time of fall are the most significant features, with height having the greatest impact on fall duration, followed by radius. Smaller spheres experienced longer fall times due to their low mass and high air resistance, especially at heights around 1000m. The analysis began with data cleaning, removing N/A or non-numerical entries, followed by plotting height vs. time, generating a correlation matrix, performing linear regression, testing gradient descent variations, and validating the models.

were included so that when `.dropna()` was used, these lines of faulty data were removed. A second round of data cleaning followed, where any non-numeric entries were converted to `'NaN'` to then be filtered out by a second `.dropna()` operation. The data set was then viewed in summarised form to check all the headings were present and correct, as shown in figure 1.

	material	density (kg/m^3)	radius (m)	mass (kg)
0	iron	7874.0	0.005	0.00412
1	iron	7874.0	0.005	0.00412
2	iron	7874.0	0.005	0.00412
3	iron	7874.0	0.005	0.00412
4	iron	7874.0	0.005	0.00412
	temperature (K)	pressure (Pa)	height (m)	time (s)
	289.6	99939.0	977	23.8
	280.8	99509.0	658	17.8
	277.9	101066.0	877	24.5
	306.8	100383.0	886	22.7
	311.3	103251.0	636	17.2

Figure 1: An initial view of the data, up to row 4.

2 Part 1

2.1 Theory and Method

2.1.1 Reading and cleaning the csv file

The data file was reading using `'panda.read_csv'` from the pandas library. Arguments designed to detect null data points and random spaces

The pandas aggregate function was used to display the maximum and minimum values of

each feature. During inspection, an anomalous entry 'm' in the 'materials' column was addressed by renaming it and using '.unique()' to verify the changes. Column names were simplified to improve code functionality, as spaces and units caused issues. A line of code with 'inplace=True' removed units from feature names, modifying the original dataframe, as preserving its initial format was unnecessary for the program's execution.

There was some difficulty found ensuring all errors were filtered out of the CSV file due to the size of the data set and the variety of errors: negative values, spaces, letters, NaN or null were inputted seemingly randomly across all columns. This made it inadequate for only the numerical columns to be filtered for non-numeric data, and required multiple filtering processes to sort out.

2.1.2 Plotting measured time vs height of fall

The imported library 'seaborn' contains a scatter plot function well suited to large data sets, so this was the obvious choice for newlinet 1b of the code. By using the 'hue=' and 'style=' arguments for secondary independent variables radius and mass respectively helped visualise the data further than a simple height vs time plot. A change in colour and marker style across the plot showed a clear relationship between radius/mass and height, as would be expected for a sphere falling in air.

By putting this plotting mechanism within a 'for' loop, looping over each unique name in the materials column (excluding the 'm', as that was removed earlier), seven plots were obtained, each one showing similar shapes for their respective data. In order for this 'for' loop to work, a filter function had to be constructed to check the index of the chosen material name and select all data that was in number form using the '.select_dtypes(include=["number"])' argument.

2.2 Results and Discussion 1

The height of the smaller radius points (i.e. more time taken to fall) denoted by the lighter markers on figures 2, 3 and 4 are surprising to notice, given that these metal spheres would experience less of a drag force as they fall in air due to their smaller surface area. However, when we consider the fact that every collision with a particle of air will exert more of a reactive force on a smaller sphere than a larger one due to its having less momentum, this result makes sense. Furthermore, if this experiment was conducted outside, as the fall heights of up to a kilometre would suggest, it's possible that gusts of wind would blow the smaller spheres further off course, extending their fall time.

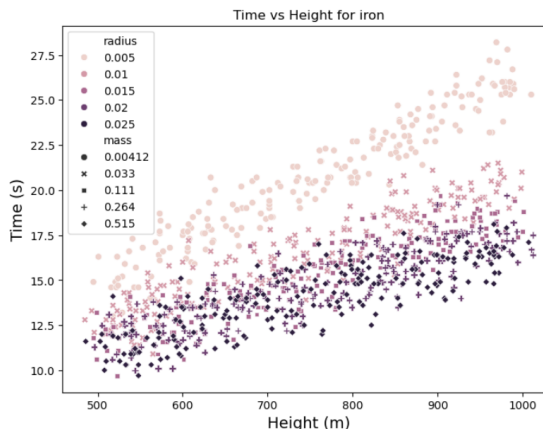


Figure 2: A height of fall vs time measured graph for all the iron data.

On inspection of the graphs, we can see that this trend of the smaller spheres falling slower applies for all materials shown, and it did for the one not shown as well. Given that the density of each material is constant, mass would be positively proportional to radius as well (denoted by the different point markers in figures 2, 3 and 4).

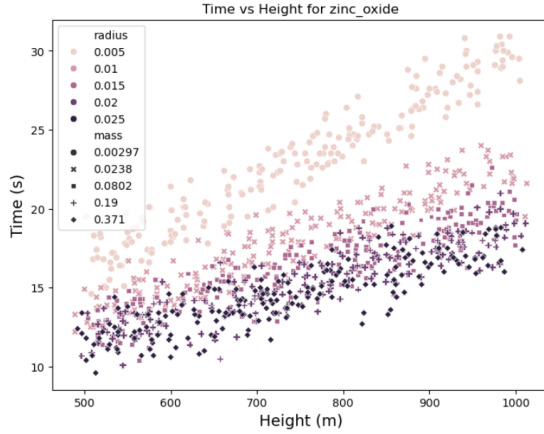


Figure 3: A height of fall vs time measured graph for all the zinc oxide data.

However, despite the lightest and smallest spheres falling slowest, we cannot assume that the largest and heaviest would fall fastest. As can be seen in figures 2 and 3 (iron and zinc oxide), there are outliers at heights of 560-600m for iron and at approx. 550m and approx. 650m for zinc oxide, where a + (indicating the second-heaviest sphere type) falls faster than any diamond marker, against expectations constructed on the majority of the data's positioning.

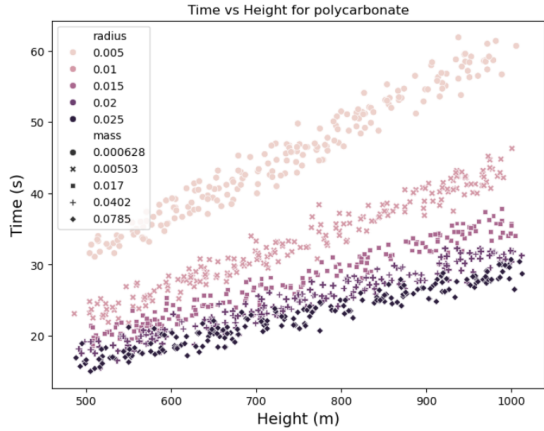


Figure 4: A height of fall vs time measured graph for all the polycarbonate data.

Figure 4 shows one case where these outliers were not present and the points are roughly organised top to bottom from smallest and lightest

to largest and heaviest.

Given that the relationship between mass and radius for a sphere has an r^3 term, the mass of a sphere increases parabolically as radius increases. Given also that the radii of the spheres used in all cases increase in 5mm increments, the mass can be expected to increase by a greater amount as the spheres get larger. Consulting the graphs, we can see that as the spheres get heavier, the general trend lines' gradients decrease by a smaller amount each time the colour of marker changes. This implies that the gradient converges to some limit at which the fall times for spheres of a heavy enough mass and large enough radius cover a smaller range on the time axis than any smaller mass and radius. The assumption is that this limit is an infinitely close approximation of the case where these spheres fall in a vacuum, and the implication of this is that a sphere of infinite radius and mass will fall as if air resistance has no effect on it whatsoever, however at this point the gravitational pull of such an object may interfere with the experimental apparatus, notably the planet on which these spheres and being dropped.

3 Part 2

3.1 Theory and Method

3.1.1 Creating a correlation matrix

The Pearson correlation function, defined in Equation (1), was implemented using the data frame's attribute `'corr()'`. This function works by summing the distances of each data point from a mean value line drawn through a feature's data and multiplying this sum by the same sum performed on another feature's data. The numerator is then the covariance of the two selected features.

This product is divided by the root of the square of the same operation, or the normalised covariance, giving a positive denominator such that the sign of the numerator may display positive

or negative correlation and the magnitude of the fraction stays between -1 and 1.

An r score near 1 or -1 shows strong positive or negative correlation respectively, and any score near 0 shows near to or no correlation. This function sorts through the data, calculating r for every possible x and y .

$$r = \frac{\sum(x_i - \bar{x}_i)\sum(y_i - \bar{y}_i)}{\sqrt{\sum(x_i - \bar{x}_i)^2\sum(y_i - \bar{y}_i)^2}} \quad (1)$$

The variable responsible for this matrix was then inputted to the arguments of a seaborn heat map and plotted using matplotlib.

3.2 Results and Discussion 2

By looking at the correlation matrix shown below, we can determine which features had the biggest impact on the fall time of the materials tested in the data frame. As shown by the colour gradient scale on the right of the figure, lighter squares indicate more positive correlation while darker squares indicate more negative.

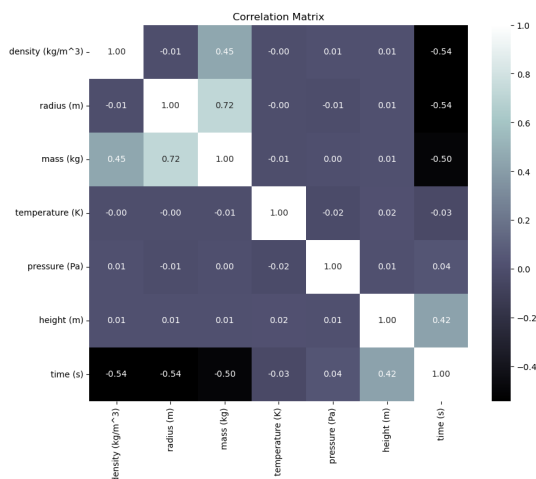


Figure 5: The correlation matrix generated from part 2 of the code, showing the correlation strengths between each of the seven features.

Starting with the most strongly time-correlated feature, height matches up to an arbitrary measure of 0.42. This makes total sense.

Interestingly, density and radius were both more negatively correlated with time than height was positively correlated, sharing a value of -0.54, closely followed by mass with -0.50. It follows that air resistance is a key factor in differentiating the fall times between different densities and radii, as a ball with a larger surface area will experience more drag as it falls, but this drag will constitute a smaller portion of the net forces acting on the ball as its mass is increased, because its weight due to gravity will be larger.

Somewhat unsurprisingly, the correlations between mass and density and mass and radius are quite strong. The lower correlation coefficient between mass and density could suggest that mass varies non-linearly with density or that density is affected by other factors, such as radius here.

There is a swathe of near-zero values in the matrix, mostly made of the correlation coefficients between temperature and pressure and the other features, showing that although values for temperature and pressure were faithfully recorded for every data point, they did not have much of an effect on any measurement.

4 Part 3

4.1 Theory and Method

4.1.1 Plotting a regression line for measured time vs variable of choice

For the sake of visualisation, a cell was included to plot a variable of choice against time with a linear regression model fitted to the two features' shape on a two-dimensional plot. Once again, a for loop was implemented with a line checking the index of each material and selecting the corresponding data from the csv file. The variable x was set outside of the loop such that the independent variable plotted against time could be changed to suit the user's interest. The 'model' variable was then assigned to the 'LinearRegres-

sion()' function and given the attribute '.fit(x, y)' to perform the regression fitting.

Finally, a scatter plot was made using matplotlib for the variables x and y to display this fitting for each material.

4.1.2 Finding the MSE coefficients

The standard least-squares regression analysis uses the Mean Square Error (MSE) loss function, defined below in equation (2). This function sums the squares of the vertical distances of each point of a feature from a linear regression line (as seen above, but for all variables at once instead of one at a time) and divides by the number of points there are for that feature, thus calculating the mean of the squared errors.

$$MSE = \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{N} \quad (2)$$

The dependent variable y was a function of the six features excluding time, and the beta coefficient for each feature was found according to equation (3):

$$y_i = \beta_0 + \sum_{j=1}^N \beta_j X_{ij} + \epsilon_i \quad (3)$$

In order to repeat this analysis for different functions, the model used was simply changed from an MSE linear regression to a Mean Absolute Error (MAE) model, a lasso or a ridge style regression model.

4.2 Results and Discussion 3

4.2.1 Using regression models and loss functions

Starting the results for part three with a linear regression for height of fall vs measured time for iron only, it is evident that the majority of data was grouped below the line, but the height of the smaller/smallest radius data points above the rest (c.f. figures 2, 3 and 4) has skewed the

regression line towards one with a steeper gradient. It is possible that if spheres with even smaller and larger radii to values of, for example, 1mm at the lower end and 1m at the larger end, this regression line would become flatter.

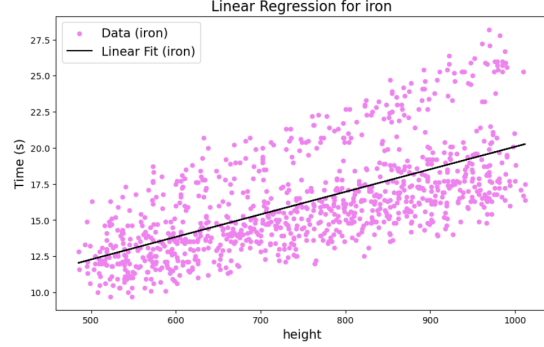


Figure 6: A linear regression plot for the scatter graph of measured time vs height of fall for iron only.

Figure 7 shows a similar regression line but for spheres grouped by their radii, again only for iron. Both height and radius plots for all other materials looked very similar to figures 6 and 7, with no distinct difference in shape.

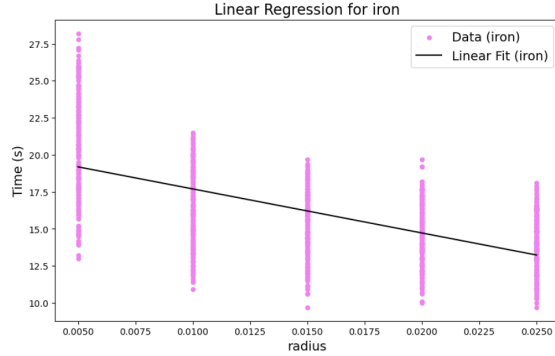


Figure 7: A linear regression plot for the scatter graph of measured time vs radius of ball for iron only.

Upon analysis of the beta coefficients produced by each of the four regression methods used, figures 8 to 11 were produced. These four methods were: Standard linear regression, ridge regression, lasso regression and Huber loss, which used mean absolute error as a loss function to be less tolerant of outliers in the data set. In

each regression mode, the beta coefficients signified the weighting of each independent variable as predictors of the dependent variable, which was time. That is, these coefficients quantified the magnitude and direction of the correlation between the independent variables and the dependent/target variable.

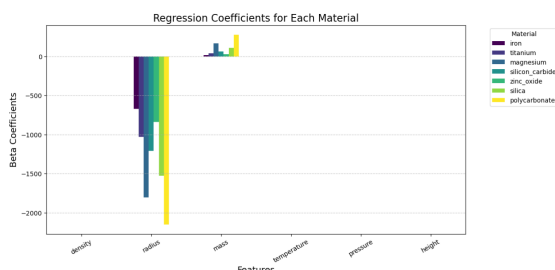


Figure 8: A bar chart for the beta coefficients generated by the linear regression MSE model. Colour of bar indicates the material for which each beta coefficient was calculated.

For the linear regression model above, the radius was shown to be the heaviest weighted predictor for fall time, with a negative quantity showing that radius is inversely proportional to fall time (so a smaller sphere falls for longer, as has already been shown and discussed). The mass of each sphere was a lower-magnitude positive predictor, which confusingly indicates that a heavier sphere falls for longer, despite what was just discussed regarding radius. However, given the difference in magnitude of these groups of beta coefficients, it is clear that these two effects work against each other as a sphere falls, but the radius prevails as the main predictor of fall time. As is seen in figures 9 and 10 just as is here in figure 8, the beta coefficients for polycarbonate were the biggest in magnitude. For all features where the beta coefficients appear not to have been added to the bar charts, the coefficients were simply too small to see in comparison to those of the other features.

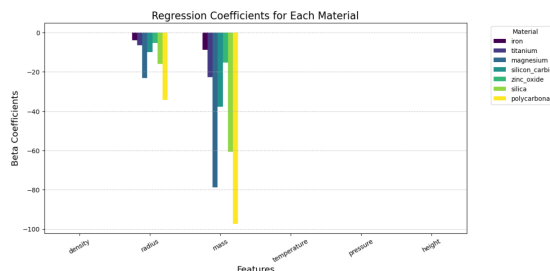


Figure 9: A bar chart for the beta coefficients generated by the Ridge MSE regression model. Colour of bar indicates the material for which each beta coefficient was calculated.

For the ridge regression model in figure 9, the mass was shown to be the heaviest weighted predictor for fall time, with a negative quantity showing that mass is inversely proportional to fall time, to the contrary of the linear regression shown in figure 8. The magnitude of the greatest beta coefficient for this figure is about twenty times smaller than that of figure 8 as well, highlighting the great effect that a different model can have on the loss function and beta coefficients. The radius coefficients were even smaller than those for mass in figure 9, so at least these models agree that radius inversely affects fall time.

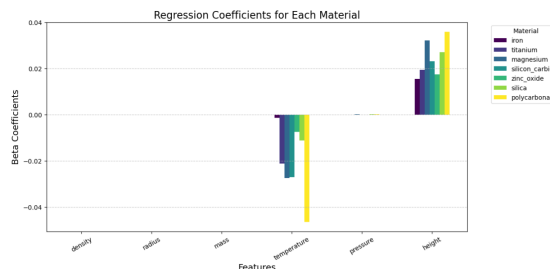


Figure 10: A bar chart for the beta coefficients generated by the Lasso MSE regression model. Colour of bar indicates the material for which each beta coefficient was calculated.

On the contrary, figure 10's beta coefficients for radius were negligible, just like density, mass and to a lesser extent pressure. The temperature coefficients here are basically nonsensical, as temperature has effectively zero correlation with

time, as shown by figure 2 (the correlation matrix). The positive prediction strength of height makes perfect sense physically.

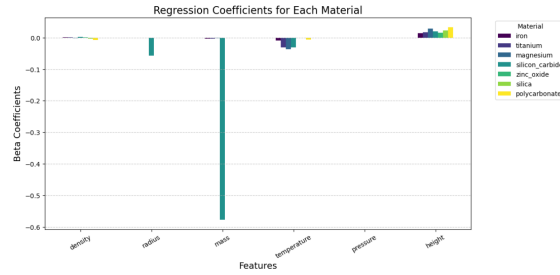


Figure 11: A bar chart for the beta coefficients generated by the Huber Regressor Mean Absolute Error (MAE) model. Colour of bar indicates the material for which each beta coefficient was calculated.

For the MAE model, mass was the greatest predictor of fall time, showing that a large mass predicts a short fall time, as discussed earlier.

Figures 12 to 15 show the values of the loss functions for each model and for each material. These are included purely to point out the very similar structures across different models, showing that the difference between loss function values is being affected by the material properties of each material rather than some difference in the calculation processes of each.

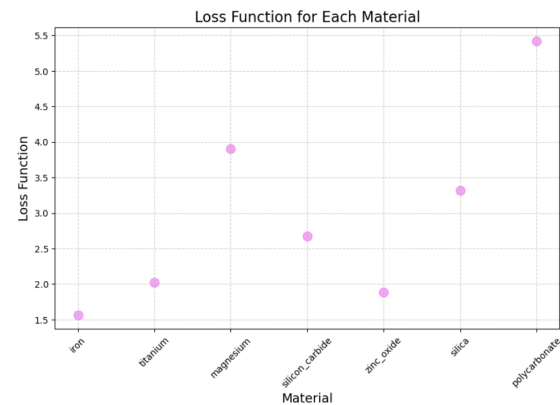


Figure 12: A scatter graph showing the values of the Linear Regression style MSE loss function for each material in the data set.

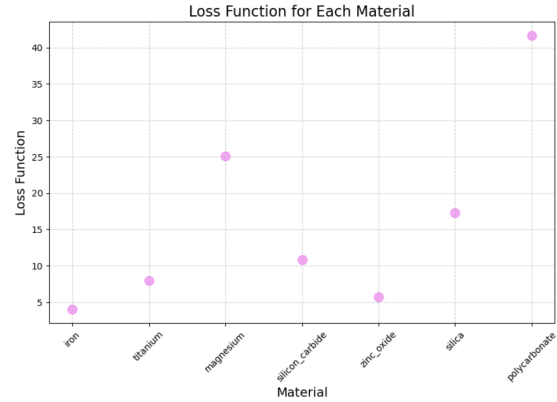


Figure 13: A scatter graph showing the values of the Ridge Regression style MSE loss function for each material in the data set.

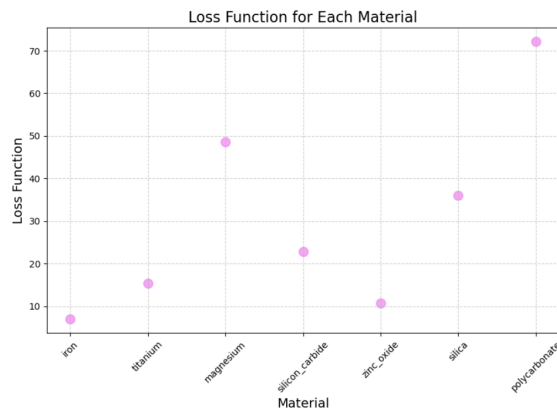


Figure 14: A scatter graph showing the values of the Lasso Regression style MSE loss function for each material in the data set.

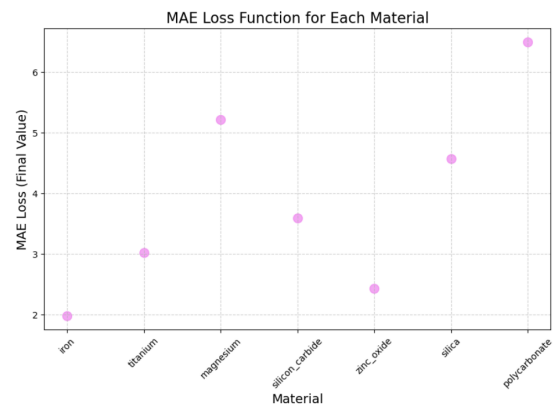


Figure 15: A scatter graph showing the values of the

Huber Regressor style MAE loss function for each material in the data set.

4.2.2 Standard batch gradient descent

The optimal learning curve found through experimentation is shown in figure 16. The learning rate of 0.003 was accommodated by a 1000 step process, reaching a loss function value of 2.2484 (5 s.f.) with a percentage improvement compared to starting value of 99.02%.

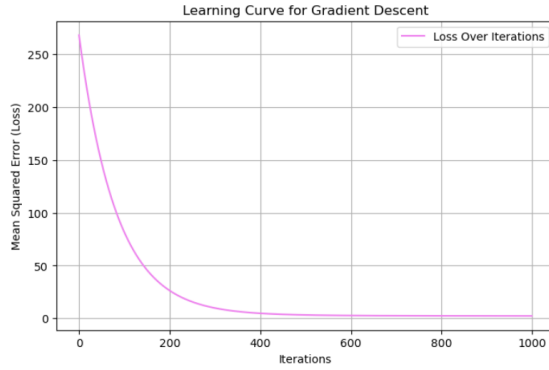


Figure 16: Learning curve drawn for standard batch gradient descent with learning rate of 0.003 and 1000 steps.

For experimentation purposes, the learning was decreased by a factor of ten to 0.0003 with the steps kept at 1000. This is displayed in figure 17, below. Obviously, 1000 steps were not enough for this model to reach a valid approximation for the loss function, as the final calculated value was 82.747 (5 s.f.) and the percentage improvement was only 63.96%.

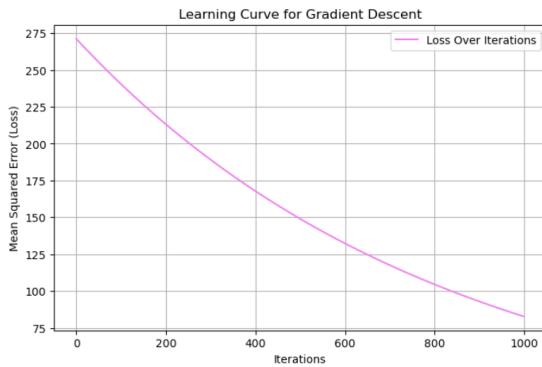


Figure 17: Learning curve drawn for standard batch gradient descent with learning rate of 0.0003 and 1000 steps.

4.2.3 Stochastic gradient descent

For stochastic gradient descent under the same learning rates and step count, the first case yielded a result nearly as accurate as the standard method, with a loss function value of 0.85376 (5 s.f.) and an improvement percentage of 91.95%, shown in figure 18.

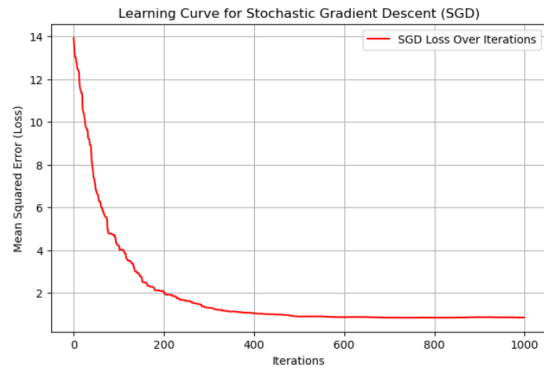


Figure 18: Learning curve drawn for stochastic gradient descent with learning rate of 0.003 and 1000 steps.

Figure 19 shows a similarly unsuccessful learning curve to figure 17, with learning rate decreased to 0.0003. The final loss was 4.0559 (5 s.f.) and improvement of only 61.76%.

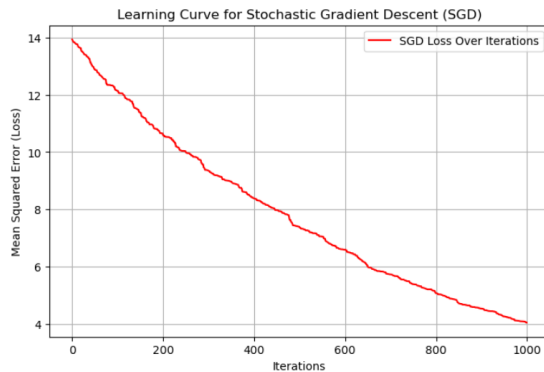


Figure 19: Learning curve drawn for stochastic gradient descent with learning rate of 0.0003 and 1000 steps.

So far, standard gradient descent wins over stochastic as it is more accurate with the same learning rate and steps, and also computes an answer in a shorter time (run times were not included here).

4.2.4 Mini-batch gradient descent

The same parameters for eta and steps as above were used in the cases of figures 20 and 21, with the higher learning rate of 0.003 yielding a final loss of 0.84326 (5 s.f.) and improvement of 92.05%. These values show slight improvements of those for the more accurate stochastic case shown in figure 18.

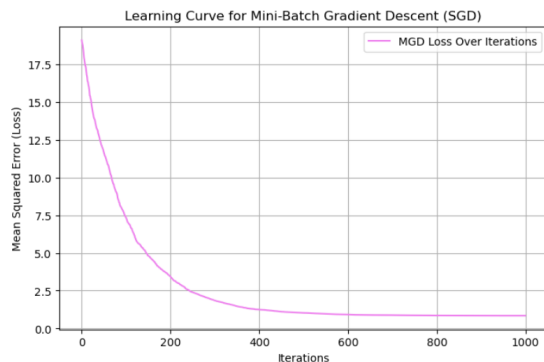


Figure 20: Learning curve drawn for mini-batch gradient descent with learning rate 0.003 and 1000 steps.

As has been a constant so far, a smaller learning rate of 0.0003 produces a less accurate final value. This was 8.0788 for this case, which is between the values for stochastic and standard gradient descent. At an improvement of 23.84%, this implies that mini-batch gradient descent is less efficient at the same number of steps as the other methods.

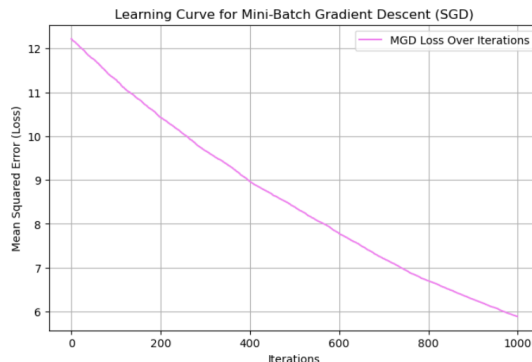


Figure 21: Learning curve drawn for mini-batch gradient descent with learning rate 0.0003 and 1000 steps.

However, based on the fact that the final result for mini-batch gradient descent is more accurate than the stochastic case, it was surmised that the order of efficiency and accuracy runs thus: standard batch style, mini-batch style, stochastic style.

5 Part 4

5.1 Theory and Method

This method involved verifying the linear model within and outside the available data range. The dataset was randomly split into training (90%) and test (10%) subsets to avoid bias. The linear model was fitted to the training data, and its accuracy was tested using residuals from said data. Outside the data range, the model was compared with an analytical model for a falling body with a quadratic-style drag coefficient. The findings were analysed to evaluate the model's performance and limitations.

5.2 Results and Discussion 4

5.2.1 Verifying the model within the data range using a residual plot

The residual plot drawn up to assess the validity of the model constructed in part 3, showing the differences between the observed and predicted values on the vertical axis plotted against the

predicted values on the horizontal axis. There is a clear heteroscedasticity here, as the spread of points changes across the plot. This suggests that the model's accuracy in prediction varies across the independent variables, which would be cause to assume that homoscedasticity is violated for the model. This is problematic because homoscedasticity is a required assumption for all regression methods.

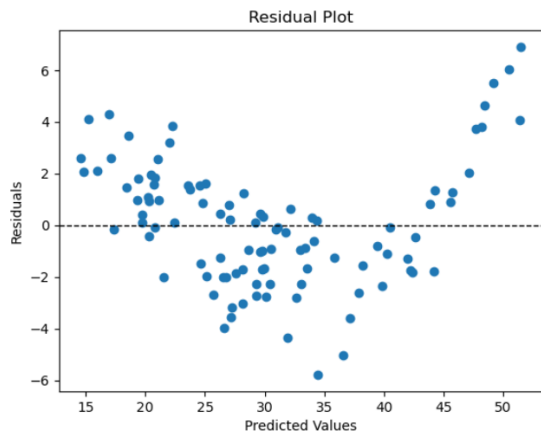


Figure 22: A plot of the residual for each point in the data frame against the predicted value for each point across the seven features.

By analysing this graph (figure 23) we see that the linear model cannot by definition match the analytical plot at heights below 200m, as the analytical plot curves down to zero. This is due to the drag effect created by air resistance causing the analytical line to flatten out for larger heights, where a sphere falling from higher ground will drop further behind a theoretical sphere dropped in a vacuum from the same height, as it has more air to travel through on its

journey to the ground. Between heights of 200m and 475m (the minimum for the CSV file's data), however, both models agree to a high degree.

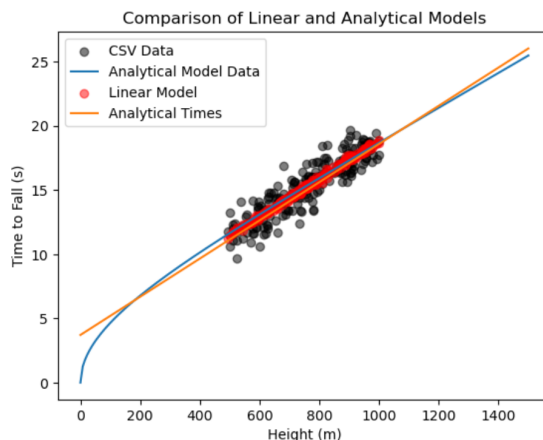


Figure 23: This graph compares the time taken for a sphere to fall from various heights, as predicted by an analytical model, a linear regression model, and actual experimental data. The black dots represent the experimental data from the CSV file, the blue line shows the analytical model's predictions, the red dots depict the linear regression model's predictions, and the orange line represents the analytical times derived from the model.

Above heights of 1021m (the maximum for the CSV file), both models generally agree and actually meet at around 1100m. Past this, they appear to diverge, as is expected due to the curve in the analytical model causing the linear fit to attempt to match not only the near-straight majority of the analytical model's plot but also the curved tail-end near the origin.